

口哨检测与 Set → Playing 状态转换指南

目录

- 1. [功能概述](#)
- 2. [关键参数配置（官方给出的默认值）](#)
- 3. [问题：为什么吹口哨没反应？](#)
- 4. [关键代码位置](#)
- 5. [SimRobot 中测试口哨检测](#)

1. 功能概述

在 RoboCup SPL 比赛中，机器人需要通过口哨声来触发从 Set 状态到 Playing 状态的转换。B-Human 代码支持两种方式触发这个转换：

- **GameController 发送 PLAYING 指令**
- **机器人检测到口哨声**

两种方式可以同时生效，代码注释明确写道：

“Whistle detection for waitFor* states - always active regardless of GameController status. This allows both whistle and GameController to trigger the transition to playing”

2. 关键参数配置（官方给出的默认值）

修改后的参数见3.5节

配置文件： Config/Scenarios/Default/gameStateProvider.cfg

参数	默认值	含义
minVotersForWhistle	5	**己方队伍（一支队伍）**最少需要多少台机器人参与投票

参数	默认值	含义
minWhistleAverageConfidence	1.15	平均置信度阈值，需要 > 此值才认为检测到口哨
maxWhistleTimeDifference	1000ms	多台机器人检测到口哨的时间窗口
ignoreWhistleAfterKickOff	3500ms	开球后忽略口哨的时间（避免误检）

3. 问题：为什么吹口哨没反应？

3.1 问题现象

SimRobot 7v7 测试时： 口哨检测正常工作，观察到的数据：

- confidenceOfLastWhistleDetection = 2
- channelsUsedForWhistleDetection = 2 或 4
- lastTimeWhistleDetected 显示时间戳，正常更新

但是我们**日常训练3v3时候的状况**：（因为机器人个数不够，只能3vs3,笑死）吹口哨后机器人没有从 Set 状态进入 Playing 状态。

这说明口哨检测本身是工作的，问题可能出在 **投票人数不足** 导致的**稀释**机制。

3.2 "稀释置信度"是什么意思？

核心问题： 代码第 598-602 行的逻辑。详细代码见第4部分

```
if(data.size() < minVotersForWhistle && data.size() < numOfVoters)
{
    const size_t missing = minVotersForWhistle - data.size();
    numOfChannels += static_cast<int>(missing * numOfMissingChannels / (num
```

这段代码的意思是：

- 如果听到口哨的机器人数量 < 5（minVotersForWhistle）
- 并且听到口哨的机器人数量 < 场上总机器人数量
- 那么，把"没听到口哨的机器人的通道数"加到分母 numOfChannels 上

- 满足上边的情况就会**稀释**

举个例子说明

因为**minVotersForWhistle默认值**是5，假设 3v3 训练，3 台机器人都听到了口哨(也可能3台没有全部都听到):

- 每台置信度 = 2，通道数 = 2
- $\text{totalConfidence} = 2 \times 2 + 2 \times 2 + 2 \times 2 = 12$
- $\text{numOfChannels} = 2 + 2 + 2 = 6$

代码会稀释！因为 $\text{data.size()}=3 < \text{minVotersForWhistle}=5$

```
missing = 5 - 3 = 2 // 还差2台机器人
numOfChannels += 2 * (假设每台2通道) = 4 // 分母增加4
```

最终：

```
numOfChannels = 6 + 4 = 10
12 / 10 = 1.2 > 1.15 ✓ 勉强通过
```

如果遇到一下情况，尽管达到了置信度可能还是没反应：

1. 不是所有3台都听到了口哨

- 如果只有1-2台听到，稀释后可能不够
- 例如：只有1台听到，置信度=2，通道=2

```
totalConfidence = 2 × 2 = 4
numOfChannels = 2 + (4台没听到 × 2通道) = 10
4 / 10 = 0.4 < 1.15 × 不通过！
```

2. useGameControllerData 参数的影响

- 如果 GC 认为场上有 5 台机器人 (5v5 模式)，但只有 3 台通信
- 代码会认为有 2 台"失联"，把这 2 台的通道数也加到分母上

3. 时间窗口问题

- $\text{maxWhistleTimeDifference} = 1000\text{ms}$
- 如果 3 台机器人检测到口哨的时间差 > 1 秒，不会被一起计算

3.4 根本原因总结

分母被"人为"增大了

代码的设计逻辑是：

- 如果场上应该有 5 台机器人，但只有 3 台听到口哨
- 代码会假设那 2 台"没听到"的机器人也在监听
- 把他们的通道数加到分母上，相当于"他们投了反对票"
- 这样可以防止少数机器人误判导致全队误动作

但这个设计在训练时（机器人数量不足5台）会导致问题

3.5 解决方案

修改 Config/Scenarios/Default/gameStateProvider.cfg：

```
# 方案1：降低最小投票人数（推荐用于训练）
minVotersForWhistle = 1; # 单台机器人就能触发
minWhistleAverageConfidence=0.5-1; #降低平均置信度阈值
```

```
# 方案2：根据训练人数设置
minVotersForWhistle = 3; # 3v3 训练时使用
minWhistleAverageConfidence=0.5-1; #降低平均置信度阈值
```

推荐方案1，把 minVotersForWhistle 改成 1，这样：

- 只要 1 台机器人听到口哨且置信度够，就能触发
- 不会有"稀释"问题
- 训练和比赛都能用（比赛时多台机器人听到只会更可靠）

4. 关键代码位置

4.1 状态转换逻辑

文件：Src/Modules/Infrastructure/GameStateProvider/GameStateProvider.cpp
第 253-304 行：

```
// Whistle detection for waitFor* states - always active regardless of Game
// This allows both whistle and GameController to trigger the transition to
switch(gameState.state)
```

```

{
    case GameState::waitForOwnKickOff:
    case GameState::waitForOpponentKickOff:
    case GameState::waitForDropBall:
    case GameState::waitForOwnPenaltyKick:
    case GameState::waitForOpponentPenaltyKick:
    case GameState::waitForOwnPenaltyShot:
    case GameState::waitForOpponentPenaltyShot:
    if(checkForWhistle(minWhistleTimestamp, gameState.gameControllerActive)
    {
        // 切换到对应的 playing 状态
        switch(gameState.state)
        {
            case GameState::waitForOwnKickOff:
                gameState.state = GameState::ownKickOff;
                break;
            case GameState::waitForOpponentKickOff:
                gameState.state = GameState::opponentKickOff;
                break;
// ... 其他状态
        }
        gameState.whistled = true;
    }
    break;
}

```

4.2 口哨检测函数详解

文件：Src/Modules/Infrastructure/GameStateProvider/GameStateProvider.cpp，
第 558-625 行

完整代码：

```

bool GameStateProvider::checkForWhistle(unsigned from, bool useGameContro
{
    std::vector<const Whistle*> data; // 存储听到口哨的机器人数据
    int numOfChannels = 0; // 总麦克风通道数（分母）
    int numOfMissingChannels = 0; // 没听到口哨的机器人的通道数
    size_t numOfVoters = 0; // 参与投票的机器人总数
    // ===== 第一步：统计自己 =====
    if(theWhistle.channelsUsedForWhistleDetection > 0)
    {
        ++numOfVoters; // 自己算一个投票者
    }
}

```

```

    numOfChannels += theWhistle.channelsUsedForWhistleDetection; // 加上自己
    if(theWhistle.lastTimeWhistleDetected > from) // 如果自己听到了口哨
        data.emplace_back(&theWhistle); // 把自己加入"听到口哨"的列表
}
// ===== 第二步: 统计队友 =====
for(const Teammate& teammate : theTeamData.teammates)
    if(teammate.theWhistle.channelsUsedForWhistleDetection > 0)
    {
        ++numOfVoters; // 队友算一个投票者
        if(teammate.theWhistle.lastTimeWhistleDetected > from) // 如果队友听到了口哨
        {
            numOfChannels += teammate.theWhistle.channelsUsedForWhistleDetection;
            data.emplace_back(&teammate.theWhistle); // 加入"听到口哨"列表
        }
        else // 队友没听到口哨
            numOfMissingChannels += teammate.theWhistle.channelsUsedForWhistleDetection;
    }
// ===== 第三步: 如果用GC数据, 补充未通信的队友 =====
if(useGameControllerData)
{
    // 从GC获取己方未被罚下的球员数
    size_t numOfPlayers = 0;
    const RoboCup::TeamInfo& ownTeam = theGameControllerData.teams[...];
    for(int i = 0; i < MAX_NUM_PLAYERS; ++i)
        if(ownTeam.players[i].penalty == PENALTY_NONE)
            ++numOfPlayers;
    // 计算"失联"的队友数 (GC说有5人, 但只收到2人通信, 则有2人失联)
    const size_t numOfMissingPlayers = numOfPlayers - std::min(numOfPlayers, numOfVoters);
    numOfMissingChannels += numOfMissingPlayers;
    numOfMissingChannels += static_cast<int>(2 * numOfMissingPlayers); // 作
}
// ===== 第四步: 关键! 稀释置信度 =====
if(data.size() < minVotersForWhistle && data.size() < numOfVoters)
{
    // 听到口哨的人数 < 5, 且 < 总投票人数
    // 则假设那些"没听到"的人也在监听, 把他们的通道数加到分母上
    const size_t missing = minVotersForWhistle - data.size();
    numOfChannels += static_cast<int>(missing * numOfMissingChannels / (numOfVoters + missing));
}
// ===== 第五步: 计算并判断 =====
// 按时间排序
std::sort(data.begin(), data.end(), ...);

for(size_t i = 0; i < data.size(); ++i)
{
    float totalConfidence = 0.f;

```

```
for(size_t j = i; j < data.size(); ++j)
{
    // 累加置信度（置信度 × 通道数）
    totalConfidence += data[j]->confidenceOfLastWhistleDetection
    * static_cast<float>(data[j]->channelsUsedForWhistleDetection);

    // 最终判断: totalConfidence / numOfChannels > 1.15 ?
    if(totalConfidence / numOfChannels > minWhistleAverageConfidence)
        return true; // 检测到口哨!
}
return false; // 没检测到
}
```

逐步解释：

变量含义：

变量	含义	例子
data	听到口哨的机器人列表	3台机器人中有2台听到 → data.size()=2
numOfChannels	总麦克风通道数（计算时的分母）	每台机器人2通道，3台=6通道
numOfMissingChannels	没听到口哨的机器人的通道数	1台没听到，2通道
numOfVoters	参与投票的机器人总数	3台机器人 → numOfVoters=3

最终判断公式：

$$\text{totalConfidence} / \text{numOfChannels} > \text{minWhistleAverageConfidence} \ (1.15)$$

即：（置信度 × 通道数的总和） / 总通道数 > 1.15

5. SimRobot 中测试口哨检测

5.1 测试步骤

- 1. 启动 SimRobot(路径/Build/Linux/SimRobot/Develop)
- 2. 选择一个.ros3文件
- 3. 我目前用的是 (/Config/Scenes/OtherScenes/OneTeamReferee.con) (用来训练手势识别的。也可以用前一个目录的.ros3文档)
- 4. 随即选一个robot1/data/audio/representation/Whistle
- 5. 执行 ar true (启用自主裁判模式)
- 6. 执行 GameState off (关闭 GC 数据接收)
- 7. 因为在simrobot当中，gc playing发布之后，gc端和口哨是同时发布的。既然要测试口哨，所以要执行5/6指令关闭gc端
- 8. 确保机器人处于 Set 状态 (waitForOwnKickOff 等)
- 9. 在终端输入 gc playing
- 10. 观察 Whistle 和 GameState 的变化

5.2 需要查看的 Representation

在 SimRobot 左侧边栏展开机器人节点，查看：

Representation	关键字段	说明
Whistle	confidenceOfLastWhistleDetection	置信度，需要 > 1.15 , 一般显示为2.0
Whistle	channelsUsedForWhistleDetection	麦克风通道数，应该 > 0 ,一般变为2/4
Whistle	lastTimeWhistleDetected	最后检测到口哨的时间戳
GameState	state	当前状态 (waitForOwnKickOff → ownKickOff)
GameState	whistled	是否通过口哨触发了状态转换